

 fsantanna / pico-sdl 



 **Code**  Issues 11  Pull requests 3  Agents  Actions

  main  **pico-sdl** / lua / doc
/ **guide.md** 

 **fsantanna** ! layer : text : height -> dim 53cf3f7 · 2 days ago 

Preview Code Blame

 Raw  

Guide

- [1. Introduction](#)
- [2. Initialization](#)
- [3. Basic Drawing](#)
- [4. Positioning: Modes & Anchors](#)
- [5. Advanced Views](#)
- [6. Events](#)
- [7. Layers](#)
- [8. Expert Mode](#)
- [9. Auxiliary Functions](#)
- [10. Extras](#)

1. Introduction

pico-lua is a graphics library for 2D games and applications. It is designed around 4 groups of APIs:

- `pico.output.*` : draw shapes and layers; play sounds and videos.
- `pico.input.*` : handle events, such as time passing and key presses.
- `pico.get.*` and `pico.set.*` : manage internal library state, such as current drawing color, and window dimensions.
- `pico.layer.*` : structure graphics in hierarchical scenes.

This guide is an interactive walk-through of pico-lua . It is not meant to be exhaustive. Consult the [API](#) for completeness.

From the command line, simply execute `pico-lua` with no parameters:

```
$ pico-lua
Lua 5.4.4 Copyright (C) 1994-2022 Lua.org, PUC-Rio
>
```

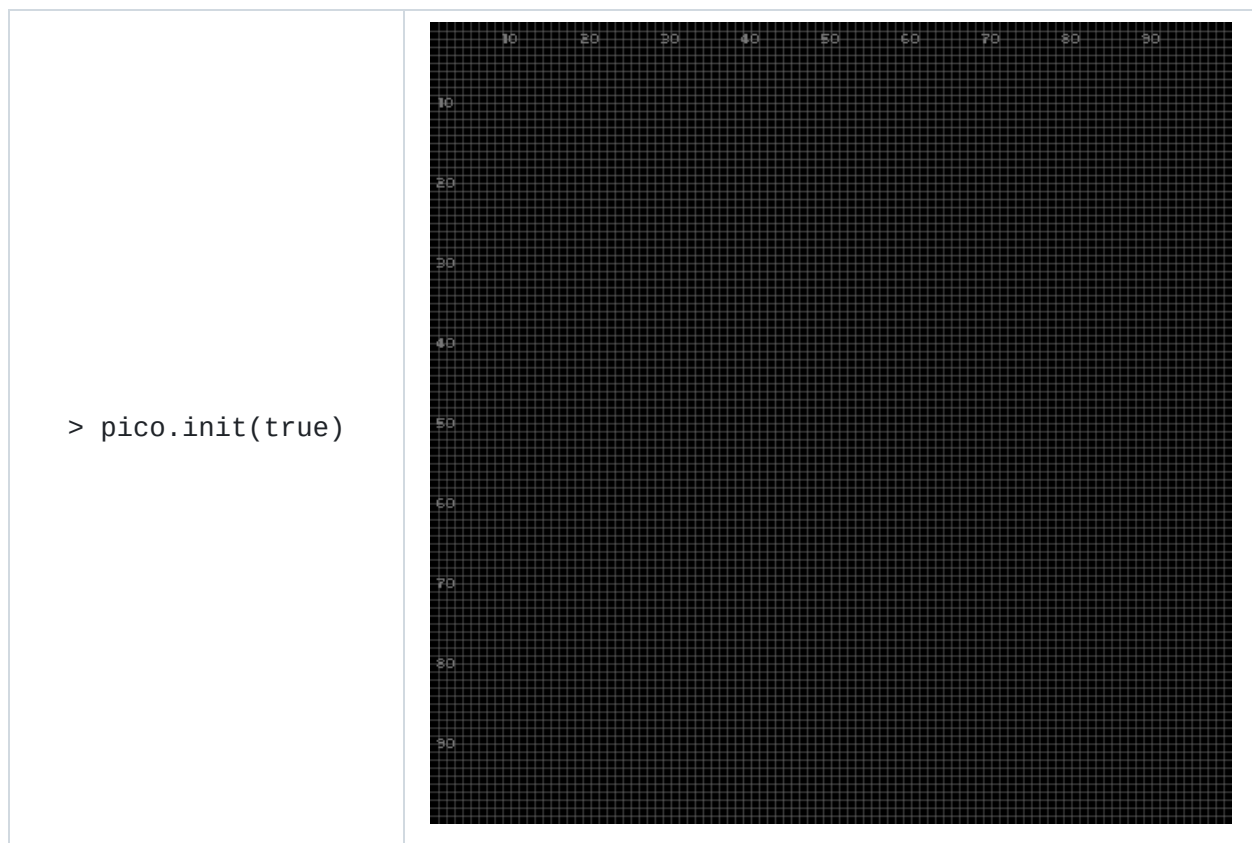


The Lua prompt `>` indicates that `pico-lua` is ready to receive commands.

2. Initialization

2.1. Open

To initialize `pico-lua`, we pass `true` to `pico.init`:



We immediately see a `500x500` **physical pixels** window divided in small `5x5` rectangles representing `100x100` **logical pixels**.

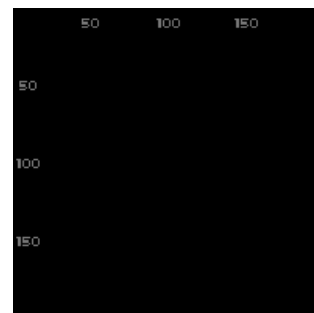
In the context of `pico-lua`, we use the term **world** to designate the logical view, which applications typically use as the main reference.

By default, note that `pico-lua` conventionally exhibits the grid and coordinate labels to aid development with visual inspection. You may click the image to zoom in.

2.2. Configure

To configure the window and the world views, we use `pico.set.scene` on the appropriate layer:

```
> pico.set.layer("window")
  pico.set.scene { dim = { '!', w=200, h=200 } }
> pico.set.layer("world")
  pico.set.scene { dim = { '!', w=200, h=200 } }
```



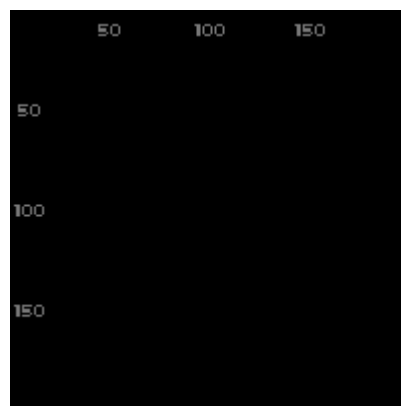
Note that `window` and `world` are pre-defined layers, which we discuss in [#Layers](#).

Using the same dimension for both the window and world, the grid disappears, since the pixel size is now `1x1`.

The character `!` indicates a dimension in "raw mode", which we discuss in [#Positioning Modes](#).

We could also use the simpler `pico.set.dim` to set the views at the same time:

```
> pico.set.dim { '!', w=200, h=200 }
```



(The image is the same as before.)

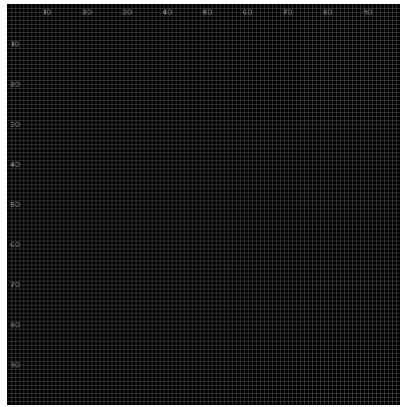
2.3. Close

In the end of the session, we should pass `false` to `pico.init` to properly finalize `pico-lua`:

```
> pico.init(false)      *(no image)*
```

Then, we can restart `pico-lua` by calling `pico.init(true)` again:

```
> pico.init(true)
```



We can see that grid and dimensions are now reset to default.

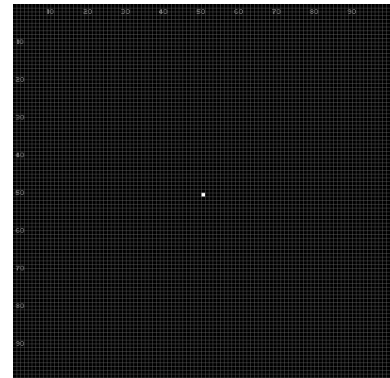
3. Basic Drawing

To ease prototyping, drawing operations in `pico-lua` take immediate effect.

3.1. Shapes

To draw a single pixel, we call `pico.output.draw.pixel` :

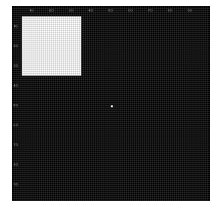
```
> pico.output.draw.pixel { '!', x=50, y=50 }
```



The pixel occupies a physical 5x5 square representing a single world pixel, as expected.

To draw a rectangle, we call `pico.output.draw.rect` :

```
> pico.output.draw.rect { '!', x=20, y=20, w=30, h=30 }
```



The table specifies a rectangle at position `(20, 20)` with size `30x30` .

Unlike most graphics libraries, `pico-lua` **centers** objects by default, which we discuss in [#Anchors](#). With the aid of the tick marks, we can see that the rectangle is actually centered at the given position.

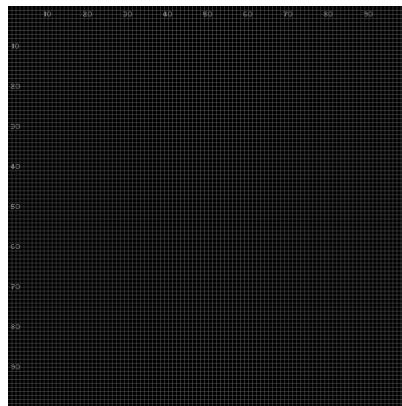
Note that `pico-lua` retains both shapes on the screen, i.e., **drawing operations are cumulative**.

Other drawing operations include `pico.output.draw.line`, `pico.output.draw.poly`, and `pico.output.draw.oval`.

3.2. Colors

To clear the screen, we call `pico.output.clear` :

```
> pico.output.clear()
```

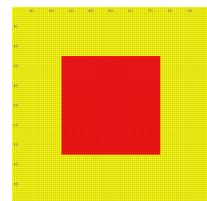


Now the pixel and rectangle are gone.

By default, `pico-lua` uses black to clear the screen and white to draw shapes.

To pick different colors, we use `pico.set.effect` and `pico.set.pencil` passing the `color` attribute:

```
> pico.set.effect { color='yellow' } -- background
> pico.set.pencil { color='red'     } -- shapes
> pico.output.clear()
> pico.output.draw.rect { '!', x=50, y=50, w=50, h=50 }
```

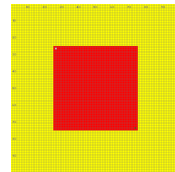


In the example, we set the background color as yellow, and the drawing color for shapes as red. **A `set` has no immediate effect on the screen, and only affects further operations**. Then, we finally clear the screen and draw a centered rectangle.

Colors can also be specified as numbers or tables:

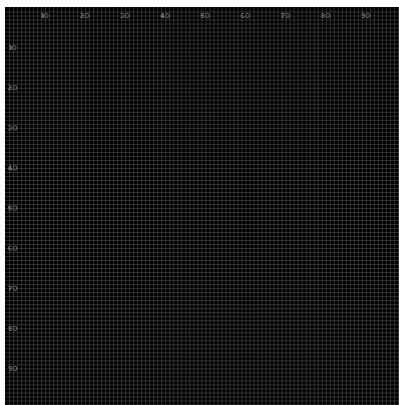
```
> pico.set.pencil {          -- 0xRRGGBB[AA]
    color = 0xCCCCCC
}
pico.output.draw.pixel { '!', x=26, y=26 }

pico.set.pencil {          -- percent mode '%' (0.0-1.0)
    color={ '%', r=0, g=0.5, b=0, a=0.5 }
}
pico.output.draw.pixel { '!', x=73, y=73 }
```



Let's restart `pico-lua` for the next examples:

```
> pico.init(false)
> pico.init(true)
```



3.3. Text, Image and Pixmap

To write text, we call `pico.output.draw.text` :

```
> pico.output.draw.text("Hello", {'!', x=50, y=33, h=30})
> pico.set.pencil { font='DejaVuSans.ttf' }
> pico.output.draw.text("Hello", {'!', x=50, y=66, h=30})
```



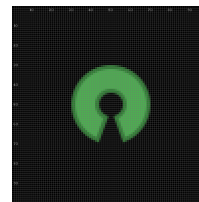
(You need to save the [font](#) on your machine.)

We first draw `Hello` at the top using the default built-in font. Then, we switch a new font and draw `Hello` again at the bottom.

Note that `pico-lua` preserves the correct text aspect ratio when we omit the width `w`.

To draw an [image](#), we call `pico.output.draw.image` :

```
> pico.output.clear()
> pico.output.draw.image("open.png", {'!', x=50, y=50})
```

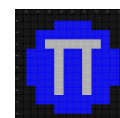


(You need to save the [image](#) on your machine.)

For images, we can omit both width `w` and height `h` to preserve original dimensions and aspect ratio.

`pico-lua` also supports **pixmaps**, which are custom arrays of colored pixels:

```
> pico.output.clear()
> _ = 'black'
  o = 'blue'
  x = 'silver'
  PI = {
    {_, _, _, o, o, o, o, _, _, _},
    {_, o, o, o, o, o, o, o, o, _},
    {_, o, x, x, x, x, x, x, o, _},
    {o, o, o, x, o, o, x, o, o, o},
    {o, o, o, x, o, o, x, o, o, o},
    {o, o, o, x, o, o, x, o, o, o},
    {o, o, o, x, o, o, x, o, o, o},
    {_, o, o, x, o, o, x, o, o, _},
    {_, o, o, o, o, o, o, o, o, _},
    {_, _, _, o, o, o, o, _, _, _},
  }
> pico.output.draw.pixmap("pi", PI, {'!', x=50, y=50, w=80})
```



The table `PI` determines the color of each pixel to draw. We use single-char color variables to initialize the pixmap array: a Greek letter `pi`, on top of a blue circle, on top of a black background.

Note that `pico.output.draw.pixmap` requires a string identifier as first parameter for proper caching.

Internally, `pico-lua` caches texts, images and pixmaps such that they are reused on subsequent redraws. We detail caching in [#Layers](#).

4. Positioning: Modes & Anchors

The **mode** determines the behavior of coordinate positions within layers:

- `'!'` - Raw: coordinates are in pixels

- '%' - Percentage: coordinates are relative to layer dimensions
- '#' - Tile: coordinates are based on tile indices

The percentage mode '%' is preferable, since it is independent of layer dimensions and adapts naturally to resizes. For instance, { '%', x=0.5, y=0.5 } always points to the center of the layer, regardless of its size.

The mode is set at index 1 in tables representing positions, dimensions, and rectangles:

- { '%', x=0.5, y=0.5 } : a centered relative position
- { '!', w=20, h=30 } : a raw dimension
- { '#', x=4, y=4, w=2, h=1 } : a rectangle spanning 2 tiles horizontally

The **anchor** determines the reference point **within** shapes:

```
+-----+
|NW   N   NE|
| W   C   E |
|SW   S   SE|
+-----+
```



When drawing, the anchor position is pinned to the given coordinate. As shown in the previous examples, by default, `pico-lua` uses the center anchor 'C'.

To specify an anchor, we use the field `anchor` in tables representing positions:

- { '%', x=0.5, y=0.5, w=0.5, h=0.5, anchor='SW' } : the rectangle's *southwest* corner pinned at center of the screen

Let's draw a pixel and three rectangles, all at the same position, but using different anchors:

```
> pico.init(false) ; pico.init(true)
> pico.set.pencil { color='white' }
  pico.output.draw.pixel { '%', x=0.5, y=0.5 }
> pico.set.pencil { color=pico.color.alpha('red', 0x80) }
  pico.output.draw.rect { '%', x=0.5, y=0.5, w=0.3, h=0.3, anchor='NW' }
> pico.set.pencil { color=pico.color.alpha('green', 0x80) }
  pico.output.draw.rect { '%', x=0.5, y=0.5, w=0.3, h=0.3, anchor='C' }
> pico.set.pencil { color=pico.color.alpha('blue', 0x80) }
  pico.output.draw.rect { '%', x=0.5, y=0.5, w=0.3, h=0.3, anchor='SE' }
```

The pixel represents the reference position $(0.5, 0.5)$ used by the three rectangles. The 0×80 alpha makes the overlapping regions reveal how the pinned anchors offset each rectangle from the same coordinate:

- the **red** rectangle places its NW corner at $(0.5, 0.5)$
- the **green** one places its center c there
- the **blue** one places its SE corner there

4.1. Tiles

The tile mode `'#'` uses discrete sized blocks as references for positions and dimensions:

```
> pico.init(false) ; pico.init(true)
> tile = { w=20, h=20 }           -- tile size (20x20 physical)
> pico.set.layer("window")
  pico.set.scene {
    tile = tile,
    dim  = { '#', w=40, h=40 }, -- window size (40x40 x 20x20 tiles)
  }
> pico.set.layer("world")
  pico.set.scene {
    tile = tile,
    dim  = { '#', w=5, h=5 },   -- world size in tiles (5x5 logical)
  }
> pico.output.draw.rect { '#', x=3, y=3, w=1, h=1 }
> pico.output.draw.rect { '#', x=5, y=1, w=2, h=1, anchor='NE' }
```

In the example, we set each tile to 20×20 and create a world of 5×5 tiles. Note that the window can also be specified in tiles.

We then draw two rectangles using the tile mode `'#'` :

- The first is centered at $(3, 3)$ occupying 1×1 tile (20×20 pixels).
- The second uses anchor `NE` , to properly occupy the top right of the screen with 2×1 tiles (40×20 pixels).

5. Advanced Views

The scene view controls how the logical world maps to the physical window. Next, we discuss some advanced properties for `pico.set.scene` :

Property	Description
----------	-------------

Property	Description
<code>target</code>	world to window mapping
<code>source</code>	visible world region
<code>clip</code>	world clipping region

The `target` property controls where the world is rendered within the physical window:

```
> pico.init(false) ; pico.init(true)
> pico.output.draw.image("open.png", {'%', x=0.5, y=0.5, w=0.5, h=0.5})
> pico.set.scene {
    target = { '%', x=0.8, y=0.8, w=0.3, h=0.2 },
}
```

Note how the world moves to fit a smaller target window at the bottom-right of the screen, with a slight distortion, since the new `w/h` changes the aspect ratio.

The `source` property selects which region of the world is visible:

```
> pico.set.scene {
    source = { '%', x=0.5, y=0.5, w=0.4, h=0.4 },
}
```



We now crop a small piece (40%) of the world from its center. The visual effect is to zoom in the image, since we stretch out the cropped world into the same target.

The `clip` property restricts the effect of drawing operations into a sub-region of the world:

```
> pico.init(false) ; pico.init(true)
> pico.set.scene {
    clip = { '%', x=0.5, y=0.5, w=0.25, h=0.25 },
}
> pico.output.draw.image("open.png", {'%', x=0.5, y=0.5, w=0.5, h=0.5})
```

We restrict drawing operations to a small region (25%) in the center of the world. Then, we draw the image centered, which is clipped to fit the specified area.

5.1. Zoom & Scroll

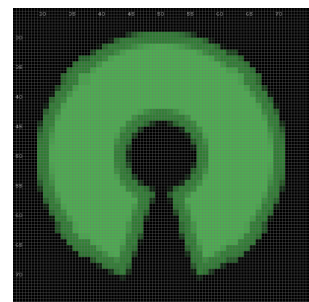
We can manipulate the `source` view property to implement not only zooming, as illustrated above, but also scrolling effects.

Let's restart with a centered image over a black window background (to match the world background when we zoom out):

```
> pico.init(false) ; pico.init(true)
> pico.set.layer("window")          -- (typically grey to distinguish fr
    pico.set.effect { color='black' }
> pico.set.layer("world")
    pico.output.draw.image("open.png", { '%', x=0.5, y=0.5, w=0.5, h=0.5})
```

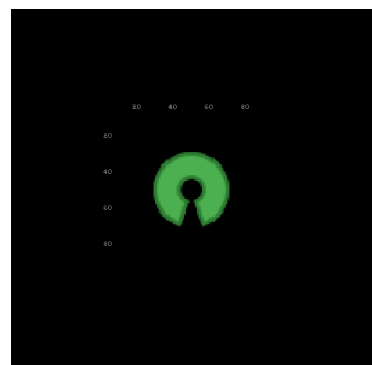
Next, we crop the view in half (`w=h=0.5`) to get a 2x zoom in:

```
> pico.set.scene {
    source = { '%', x=0.5, y=0.5, w=0.5, h=0.5 },
}
```



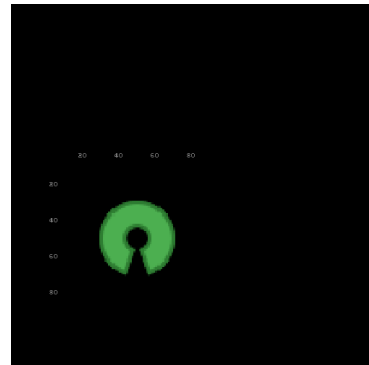
Likewise, when we double the source (`w=h=2`), we get a 1/2 zoom out:

```
> pico.set.scene {
    source = { '%', x=0.5, y=0.5, w=2, h=2 },
}
```



We can apply the same reasoning to create a scroll effect when we manipulate the `source` `x` and `y` fields:

```
> pico.set.scene {  
  source = { '%', x=0.8, y=0.2, w=2, h=2 },  
}
```



A positive $x=+0.3$ offset (from 0.5 to 0.8) crops the left side of the source, which still targets the whole window, resulting in a scroll left.

A negative $y=-0.3$ offset (from 0.5 to 0.2) crops the bottom of the source, resulting in a scroll down.

6. Events

In addition to output and drawing operations, `pico-lua` also provides functions to handle time and user input.

6.1. Delay

A call to `pico.input.delay(ms)` pauses execution for the specified time:

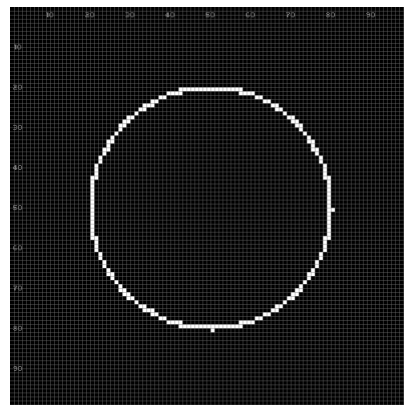
```
> pico.init(false) ; pico.init(true)  
> pico.output.draw.pixel { '%', x=0.25, y=0.5 } -- copy/paste next lines a  
pico.input.delay(1000)  
pico.output.draw.pixel { '%', x=0.50, y=0.5 }  
pico.input.delay(1000)  
pico.output.draw.pixel { '%', x=0.75, y=0.5 }
```

After each drawing, we pause execution for `1s`, also blocking (freezing) the Lua prompt.

Combined with loops, delays can create non-interactive animations.

Here, we draw a circle pixel by pixel:

```
> pico.output.clear()
> for i=1, 400 do
  local rad = i * math.pi / 180
  pico.output.draw.pixel { '%',
    x = 0.5 + 0.3*math.cos(rad),
    y = 0.5 + 0.3*math.sin(rad),
  }
  pico.input.delay(10)
end
```



On each step, we draw a single pixel and delay execution for a short period.

6.2. Event

To create interactive applications, we use `pico.input.event` to wait for input events.

Let's create a simple loop to explore the possibilities:

```
> while true do
  local e,ms = pico.input.event()
  print("EVENT", ms, e.tag)
  for k,v in pairs(e) do
    print(' ', k, v)
  end
  if e.tag=='key.dn' and e.key=='Escape' then
    break
  end
end
```



The call to `pico.input.event` returns two values:

- `e` is a table with a `tag` field that identifies the event type
- `ms` is the time elapsed between two event occurrences

You may interact with the window by pressing keys, using the mouse, resizing the window, and so on. To escape the loop, press the `Escape` key.

Event types (*tags*) include `'key.dn'`, `'mouse.button.dn'`, `'mouse.motion'`, and many others.

We can filter events and set timeouts:

```
> e1 = pico.input.event('key.dn')      -- wait for key press only  
> e2 = pico.input.event('key.dn', 5000) -- wait key up to 5000ms
```



When a timeout expires without an event, `nil` is returned:

```
> print(e2) -- nil, after 5000 if no keys pressed
```



6.3. Default Key Bindings

By default, `pico-lua` provides key bindings to zoom and scroll the current view:

- CTRL / +, - : zoom in / out
- CTRL / Arrow keys: scroll left / right

Let's draw a centered image and use the key bindings to explore it:

```
> pico.output.clear()  
> pico.output.draw.image("open.png", {'%', x=0.5, y=0.5, w=0.5, h=0.5})  
> pico.input.loop()
```

The call to `pico.input.loop()` passes full control to `pico-lua` to handle events internally.

Now, try CTRL pressing + to zoom in and the arrow keys to scroll around. Finally, close the window (e.g., ALT+F4) to escape from the loop.

7. Layers

Layers are independent views, in which you can draw shapes separately, and then compose them to form complex scenes.

The main logical world is itself a layer, as well as images, texts, pixmaps, and videos. Calls like `pico.output.draw.image(path, rect)` internally create a layer and cache it by its identifier, so repeated calls reuse it. Therefore, all discussed [#view properties](#) also apply to layers.

7.1. Creating Layers

We use `pico.layer.empty` to create a fresh layer, and `pico.set.layer` to redirect further drawing operations to it:

```
> pico.output.clear()
> pico.layer.empty { key="flag", dim={'!', w=300, h=200} }
> pico.set.layer("flag")
> pico.set.pencil { color={ r=0x00, g=0x2B, b=0x7F } }
pico.output.draw.rect { '%', x=0.00, y=0.0, w=0.33, h=1.0, anchor='N'
pico.set.pencil { color={ r=0xFC, g=0xD1, b=0x16 } }
pico.output.draw.rect { '%', x=0.33, y=0.0, w=0.34, h=1.0, anchor='N'
pico.set.pencil { color={ r=0xCE, g=0x11, b=0x26 } }
pico.output.draw.rect { '%', x=0.67, y=0.0, w=0.33, h=1.0, anchor='N'
```



At this point, nothing appears on the screen yet, since we did not update the main world view.

We identify the layer as `flag` and then set it as the current drawing layer. Then, we paint inside the layer with the colors.

Two notes about `pico.layer.empty` to be discussed further in [#Hierarchy](#):

- the first argument (`nil`) is the optional parent layer
- the third argument (`false`) determines if layer traversal automatically clears it

7.2. Compositing

To compose a layer on top of the current layer, we call `pico.output.draw.layer` :

```
> pico.set.layer("world")      -- back to the world layer
> pico.output.draw.layer("flag", {'%', x=0.33, y=0.33, w=0.2})
> pico.output.draw.layer("flag", {'%', x=0.66, y=0.66, w=0.5})
```



We first use `pico.set.layer("world")` to target the world layer. Then, we compose the flag twice, with different arguments.

We can also flip and rotate layers when compositing them, by setting their `pico.set.effect` fields:

```
> pico.set.layer("flag")
> pico.set.effect {
  flip   = 'horizontal',
  rotate = {angle=30, anchor='C'},
}
> pico.set.layer("world")
> pico.output.draw.layer("flag", {'%', x=0.75, y=0.25, w=0.3})
```



In the example, the `flip` field receives `horizontal` to reverse the stripe order. The `rotate` table takes an `angle` in degrees and an `anchor` for the pivot point.

We can also set the transparency of layers by lowering their `alpha` field:

```
> pico.set.layer("flag")
> pico.set.effect {
  rotate = {angle=0},
  flip   = 'none',
  alpha  = 0x80,
}
> pico.set.layer("world")
> pico.output.draw.layer("flag", {'%', x=0.5, y=0.5, w=0.6})
```



Following the general conventions for alpha, `0x00` is fully transparent, while `0xFF` is fully opaque.

7.3. Sub-Layers

A sub-layer points to a region within a source layer, with both sharing the actual pixel contents.

Sub-layers are useful to isolate individual frames from sprite sheets in games, which we discuss in [#Animations](#).

We call `pico.layer.sub` to crop a region of a source layer. In the next example, we want to isolate each stripe of the flag as a sub layer:

```
> pico.output.clear()
> pico.layer.sub { key="blue",   sup="flag", crop={'%', x=0.25, y=0.5, w=0
> pico.layer.sub { key="yellow", sup="flag", crop={'%', x=0.50, y=0.5, w=0
> pico.layer.sub { key="red",    sup="flag", crop={'%', x=0.75, y=0.5, w=0
> pico.output.draw.layer("blue", {'%', x=0.30, y=0.30, w=0.25})
> pico.output.draw.layer("yellow", {'%', x=0.70, y=0.45, w=0.25})
> pico.output.draw.layer("red",   {'%', x=0.45, y=0.75, w=0.25})
```

Each sub-layer crops a square from each stripe of the flag (blue, yellow, red), and then draws each on the screen.

A sub-layer uses the same APIs as layers. For instance, the second argument to `pico.layer.sub` identifies the sub-layer for further operations.

7.4. Hierarchy

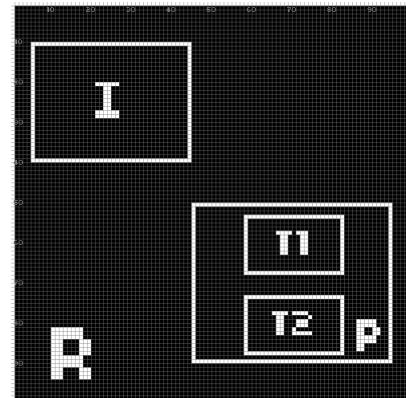
Layers in `pico-lua` can form a tree hierarchy that `pico.output.present` traverses automatically to compose the full scene onto the screen.

When creating a layer (e.g., `me`), we can pass another layer (e.g., `up`) to become its parent:

```
pico.layer.empty { up="up", key="me", ... } -- "up" is parent of "me"
pico.set.layer("me")                      -- setup "me"
pico.set.scene {                          -- position "me" within
  "up"
  target = {'%', x=0.5, y=0.5, w=0.5, h=0.5}
}
```

After we create the layer, we set the target region within its parent. In the example, the child layer `me` is centered inside parent layer `up`.

Suppose we want to draw the layout in the figure in the right: The world layer (shown as `R` in the figure) contains the image layer `I` and panel layer `P`, which contains the text layers `T1` and `T2`.



The next code listing implements this layout:

- `I`: created with `pico.layer.image` with `world` as parent
- `P`: created with `pico.layer.empty` with `world` as parent
 - `T1`: created with `pico.layer.text` with `P` as parent
 - `T2`: created with `pico.layer.text` with `P` as parent

```
> pico.init(false) ; pico.init(true)
> pico.layer.image {
  up="world", key="I", path="open.png", target={'%', x=0.3, y=0.3, w=0.4
}
> do
  pico.layer.empty { up="world", key="P", clear=true, dim={'!', w=100, h
  pico.set.layer("P")
  pico.set.effect { color='silver' }
  pico.set.scene { target = {'%', x=0.7, y=0.7, w=0.4} }
  pico.layer.text {
    up="P", key="T1", dim={'!', h=20}, text="Hello", target={'%', x=0
  }
  pico.layer.text {
    up="P", key="T2", dim={'!', h=20}, text="World!", target={'%', x=0
  }
end
```

```
> pico.output.present()
```

After the scene is composed, we simply call `pico.output.present` to update the screen all at once.

7.5. Memory Stack

Layers are memory resources that require a mechanism for consistent allocations and matching deallocations. `pico-lua` relies on a stack that provides deterministic bulk deallocations in scope transitions within applications.

The default scope opens at `pico.init(true)` and closes at `pico.init(false)`. A program can also manipulate scopes explicitly through `pico.push`, which introduces a fresh scope, and `pico.pop`, which releases all resources in the current scope.

As the next example illustrates, explicit scopes are convenient for applications that move through long sequences of scenes (e.g., `menu -> game -> game-over -> ...`). Each scene allocates its own layers under a fresh `push`, and a single `pop` releases them before the next begins, keeping memory bounded over time:

```
> pico.init(false) ; pico.init(true)
> for i=1, 5 do
    pico.push()
    pico.output.clear()
    pico.output.draw.text(i, {'%', x=0.5, y=0.5, h=0.7})
    pico.input.delay(500)
    pico.pop() -- releases implicit allocated text layer
end
```



In the example, the loop represents a continuous, possibly infinite, sequence of scenes, which would accumulate new layers per iteration. The stack not only keeps the memory footprint constant over time, but also helps to organise and reason about the application memory.

8. Expert Mode

By default, as seen in previous sections, each drawing operation becomes immediately visible on the screen.

However, to keep visual objects in perfect sync, most games and non-trivial applications require to draw them simultaneously on every frame.

For this reason, `pico-lua` supports an expert mode that buffers drawing operations until an explicit call to `pico.output.present`, which updates the screen with all objects at once:

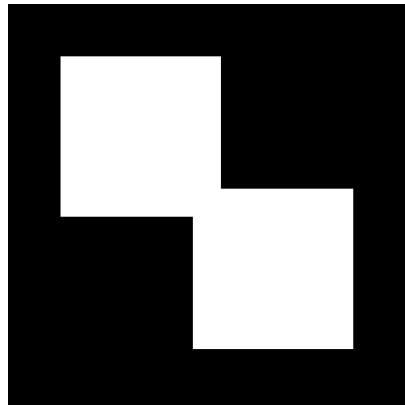
To enable the expert mode, we call `pico.set.expert`:

```
> pico.init(false) ; pico.init(true)
> pico.set.expert(true)
> pico.output.draw.rect { '%', x=0.33, y=0.33, w=0.4, h=0.4 }
pico.input.delay(1000) -- artificial delay
pico.output.draw.rect { '%', x=0.66, y=0.66, w=0.4, h=0.4 }
pico.input.delay(1000) -- artificial delay
```



Note that although the code above takes at least 2s to complete, nothing appears on the screen yet, since we have not called `pico.output.present`:

```
> pico.output.present()
```



Now, both the rectangles appear at the same time.

8.1. Application Main Loop

Typical games and graphical applications need to deal continually with moving objects, input from users, and also the passage of time. These applications follow the same structure of a continuous "main loop": poll user events, update the state of objects, redraw the whole scene (a frame), and repeat at a given *FPS rate* (frames per second).

This is what a main loop in `pico-lua` looks like:

```
-- (this is just a sketch - do not execute)

pico.set.expert(true, 40)      -- 40 FPS

while true do                  -- main loop
    pico.output.clear()
    pico.output.*()             -- scene redrawing
    pico.output.present()

    local e,dt = pico.input.event() -- event / timeout awaiting

    if e then
        (normal lua code)      -- event handler (keys, mouse)
    else
        (normal lua code)      -- timeout handler (animations,
collisions)
    end
end
end
```



The call to `pico.set.expert` serves two purposes:

1. Disables automatic screen update that would happen after every single draw call. Now, all redrawing between successive calls to `present` appears as a single frame. Even with hundreds of objects, the screen update is instantaneous.
2. Sets the FPS rate to `40`, making `pico.input.event` awake every `25ms`. If an event occurs in the meantime, say after `10ms`, then `e` is set and the next call to `pico.input.event` resumes after only `15ms` (`25-10`).

If the `FPS` argument is omitted, `pico.input.event` only awakes from events, but never from timeouts.

8.2. A Simple Example

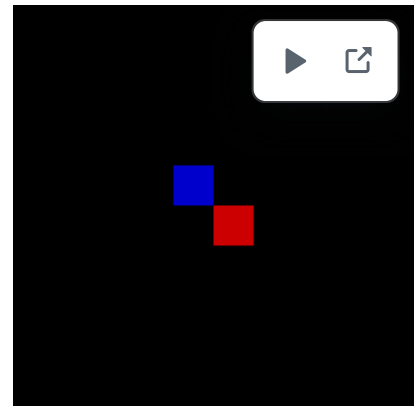
The next example tracks the position of two pixels on a `10x10` screen:

- one follows the mouse, wherever it moves to;
- the other responds to the arrow keys.

The complete source code is [here](#) (~40 lines of code).

Run the program:

```
$ pico-lua rects.lua
```



Now, let's discuss the implementation:

```
-- (omitted initialization - do not execute)

pico.set.expert(true)           -- no time

local k = {'!', x=4, y=4}       -- key pix
local m = {'!', x=5, y=5}       -- mouse p

while true do                   -- main lo
    -- redraw scene
    pico.output.clear()
    pico.set.pencil { color='red' }
    pico.output.draw.pixel(m)
    pico.set.pencil { color='blue' }
    pico.output.draw.pixel(k)
    pico.output.present()

    -- handle events
    local e = pico.input.event()
    assert(e, "no FPS set here")
    if e.tag == 'quit' then
        break
    elseif e.tag == 'mouse.motion' then
        m = pico.get.mouse '!'
    elseif e.tag == 'key.dn' then
        if e.key == 'Up' then k.y = k.y
        elseif e.key == 'Down' then k.y = k.y
        elseif e.key == 'Left' then k.x = k.x
        elseif e.key == 'Right' then k.x = k.x
        end
    end
end
```

Since this example does not contain time-based animations, we do not set FPS in `pico.set.expert`, so `pico.input.event` awaits until an event occurs.

The tables `m` and `k` represent the pixels controlled by the mouse and keyboard, respectively. They use the raw mode `'!'` and start centered at `(4,4)` and `(5,5)`.

The main loop first draws their initial positions and awaits `pico.input.event`. In this example, we only handle `quit`, `mouse.motion` and `key.dn` events:

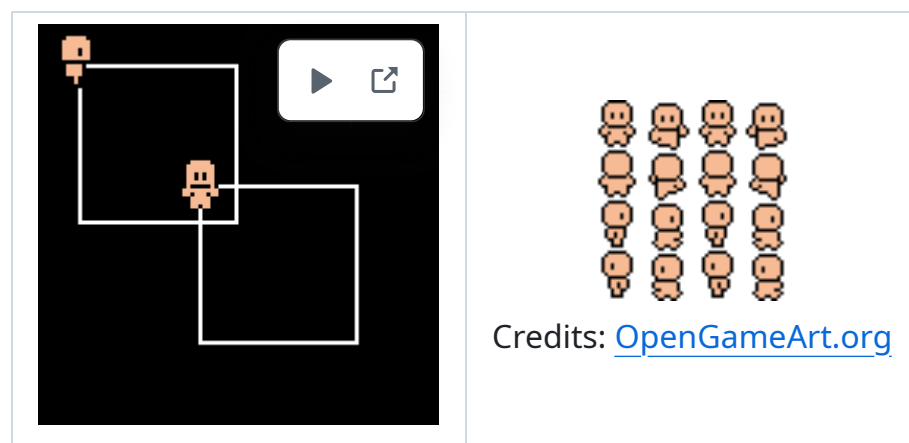
- on a quit event, we escape the loop to terminate the application;
- for the mouse, we update `m.x` and `m.y` based on the received event `e`;
- for the keyboard, we update `k.x` or `k.y` depending on `e.key`.

Then, the loop iterates to redraw the scene and wait for the next event.

8.3. Animations

The next example creates two time-based animations that require to set `pico.set.expert` with FPS.

We want the characters to move along the rectangular paths, in opposite directions, and at different speeds. The animation in the left is based on the sprite sheet in the right:



The complete source code is [here](#) (~75 lines of code).

Run the program:

```
$ pico-lua anims.lua
```



Now, let's discuss the implementation. The overall structure is always the same:

```
-- (omitted initialization - do not execute)

pico.set.expert(true, 20)

local f1, x1, y1 = walk('clock', 0, 0) -- clockwise (faster)
local f2, x2, y2 = walk('counter', 0, 0) -- counter clockwise

local t = 0

while true do -- main loop
    -- redraw the scene
    pico.output.draw.rect {
        '%', x=0.3, y=0.3, w=0.4, h=0.4 -- rect path (top-left)
    }
    pico.output.draw.rect {
        '%', x=0.6, y=0.6, w=0.4, h=0.4 -- rect path (bottom-right)
    }
    pico.output.draw.layer (
        f1, {'%', x=x1, y=y1, w=0.15} -- clockwise sprite
    )
    pico.output.draw.layer (
        f2, {'%', x=x2, y=y2, w=0.15} -- counter sprite
    )
    pico.output.present()

    -- handle events
    local e, dt = pico.input.event('quit')
    if e then
        break
    end

    t = t + dt -- handle animations
    f1, x1, y1 = walk('clock', t*2, t)
    f2, x2, y2 = walk('counter', t, t)
end
```

Now, we set FPS to 20 to animate the sprites every 50ms .

The function `walk` (see the full source) receives a clock direction and the elapsed time, returning the sprite and positions to apply.

The main loop redraws the whole scene (rectangle paths and sprites) and awaits the next clock tick. In this example, `pico.input.event` either awakes from a 'quit' event, which escapes the main loop, or after 50ms , which applies the next animation step.

In the omitted initialization, we use [#sub-layers](#) to crop the 4x4 sprite sheet above:

```

local frames = pico.layer.images {
    key="walk", path="img/walk.png", sheet={'#', w=4, h=4},
}

local dirs = {
    down = { 1, 2, 3, 4},    -- walk-01 -> walk-04
    up   = { 5, 6, 7, 8},
    right = { 9, 10, 11, 12},
    left  = {13, 14, 15, 16},
}

```

This splits the sprite sheet into layers "walk-01" to "walk-16" : walk down (01-04), up (05-08), right (09-12), left (13-16).

Then, at each loop step, the call to `walk` decodes the current state and returns the appropriate 'walk-xx' layer.

9. Auxiliary Functions

9.1. Coordinate Conversions

The set of `pico.cv.*` functions converts coordinates between modes and between layers:

```

> pos = pico.cv.pos('!', {'%', x=0.5, y=0.5})
> print(pos.x, pos.y)
50    50

```

In the example, we convert a percentage into raw coordinates, assuming the default 100x100 world layer.

Optional layer arguments can project across frames:

```

> r = pico.cv.rect('world', '%', 'window', {'!', x=250, y=250, w=100, h=100})
> print(r.x, r.y, r.w, r.h)
0.5    0.5    0.2    0.2

```

Here we project a 100x100 rectangle centered in the 500x500 window, obtaining a centered- 20% world rectangle.

`pico.cv.dim` projects dimensions the same way, without a position component:

```
> d = pico.cv.dim('world', '!', 'window', {'!', w=100, h=100})
> print(d.w, d.h)
20    20
```



9.2. Collision Detection

The set of `pico.vs.*` functions tests overlaps between points and rectangles:

```
> pico.vs.pos.rect (
    {'%', x=0.55, y=0.75},
    {'!', x=50, y=50, w=20, h=20}
)
false
```



In the example, the point at (55, 75) of the world falls outside the rectangle, which spans (40, 40) to (60, 60) .

Optional layer arguments can test across frames. When a rect side is omitted, it defaults to that layer's bounds:

```
> pico.vs.rect.rect("world", "window", {'%', x=0.5, y=0.5, w=0.5, h=0.5}, {'!', w=0.5, h=0.5})
true
```



In the example, we compare the whole world with a centered 50% region of the window, which must overlap since the world is inside the window by default.

For the sake of completion, `pico-lua` also provides `pico.vs.pos.pos` and `pico.vs.rect.pos` .

9.3. Nesting Rects

The set of `pico.xin.*` functions composes a child onto a parent rect, returning a new flat value re-expressed in the current scene:

```
> p = pico.xin.pos (
    { '%', x=0.5, y=0.5, w=0.5, h=0.5 }, -- parent
    { '%', x=0, y=1 }                    -- child
)
> print(p.x, p.y)
0.25    0.75
```



The parent rect is centered in the world with the child point at its south ($y=1$) west ($x=0$) boundary, resulting in a new world point at $y=0.75$ (south) and $x=0.25$ (west).

Nesting is useful, for instance, to check mouse collisions against shapes drawn inside shapes inside layers.

For the sake of completion, `pico-lua` also provides `pico.xin.rect` and `pico.xin.dim`.

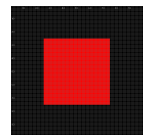
Note that the module name is `xin` because `in` is a reserved word in Lua.

10. Extras

10.1. Screenshots

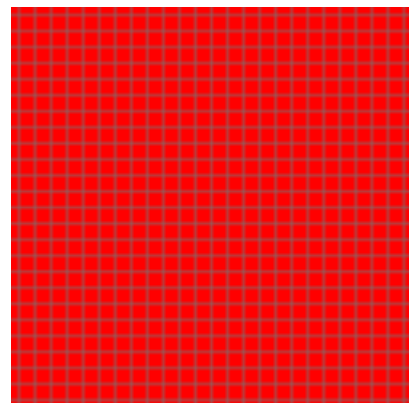
To save a screenshot of the current window, we call `pico.output.screenshot` :

```
> pico.init(false) ; pico.init(true)
> pico.set.pencil { color='red' }
> pico.output.draw.rect { '%', x=0.5, y=0.5, w=0.5, h=0.5 }
> pico.output.screenshot('window', "my-screenshot.png")
```



The first argument is the layer to capture. An optional rectangle crops the screenshot:

```
> pico.output.screenshot (
    'window',
    "crop.png",
    {'%', x=0.5, y=0.5, w=0.25, h=0.25},
)
```



We can also create a new layer from a screenshot with `pico.layer.screenshot` :

```
> pico.layer.screenshot { key="snap", sup='window' }
> pico.output.draw.layer("snap", {'%', x=0.5, y=0.5, w=0.3})
```



In the example, we freeze the `window` into `snap`, then compose it back onto the world at `30%` size.

10.2. Playing Sounds

To play a sound file:

```
> pico.output.sound("path/to/sound.wav")
```



10.3. Fullscreen Mode

To enter fullscreen mode:

```
> pico.set.window { fullscreen=true }
```



To leave fullscreen mode:

```
> pico.set.window { fullscreen=false }
```



The current fullscreen state can be queried with `pico.get.window()` :

```
> w = pico.get.window()  
> print(w.fullscreen)  
false
```

